



POO EN JAVA

TD2 : Classes et Objets

= Exercice 1 (classes et objets de Java):

1. Créez une classe "Personne" avec des propriétés telles que "nom", "prénom", "âge", etc. Ajoutez une méthode "getFullName" qui renvoie le nom complet de la personne.
2. Créez une classe "Cercle" avec une propriété "rayon". Ajoutez une méthode "calculerSurface" qui calcule et renvoie la surface du cercle.
3. Créez une classe "Rectangle" avec des propriétés "longueur" et "largeur". Ajoutez une méthode "calculerSurface" qui calcule et renvoie la surface du rectangle.
4. Créez une classe "Banque" avec des propriétés "nom" et "adresse". Ajoutez une méthode "ajouterCompte" qui permet d'ajouter un compte bancaire à la banque.
5. Créez une classe "CompteBancaire" avec des propriétés telles que "nom du titulaire", "solde", etc. Ajoutez des méthodes pour déposer et retirer de l'argent du compte.
6. Créez une classe "Voiture" avec des propriétés telles que "marque", "modèle", "année de fabrication", etc. Ajoutez une méthode "calculerAge" qui calcule et renvoie l'âge de la voiture.
7. Créez une classe "Etudiant" avec des propriétés telles que "nom", "prénom", "numéro d'étudiant", etc. Ajoutez une méthode "ajouterNote" qui permet d'ajouter une note à l'étudiant.
8. Créez une classe "Cours" avec des propriétés telles que "nom", "description", etc. Ajoutez une méthode "ajouterEtudiant" qui permet d'ajouter un étudiant au cours.
9. Créez une classe "Calculatrice" avec des méthodes pour ajouter, soustraire, multiplier et diviser deux nombres.

Exercice 2 (constructeur, getter et setter de Java)

1. Créez une classe "Personne" avec des propriétés telles que "nom", "prénom", "âge", etc. Créez un constructeur pour initialiser ces propriétés. Ajoutez des méthodes getter et setter pour chaque propriété.
2. Créez une classe "Rectangle" avec des propriétés "longueur" et "largeur". Ajoutez un constructeur pour initialiser ces propriétés. Ajoutez des méthodes getter et setter pour chaque propriété.
3. Créez une classe "CompteBancaire" avec des propriétés telles que "nom du titulaire", "solde", etc. Ajoutez un constructeur pour initialiser ces propriétés. Ajoutez des méthodes getter et setter pour chaque propriété.
4. Créez une classe "Voiture" avec des propriétés telles que "marque", "modèle", "année de fabrication", etc. Ajoutez un constructeur pour initialiser ces propriétés. Ajoutez des méthodes getter et setter pour chaque propriété.

Exercice 3 (Niveau de visibilité des champs de Java)

1. Écrivez une classe avec deux champs privés, l'un pour le nom et l'autre pour l'âge. Écrivez des

méthodes publiques pour accéder à ces champs depuis une autre classe.



2. Écrivez une classe qui contient un champ privé pour le solde d'un compte bancaire. Écrivez des méthodes publiques pour déposer et retirer de l'argent de ce compte.
3. Écrivez une classe qui contient un champ privé pour la longueur et la largeur d'un rectangle. Écrivez des méthodes publiques pour calculer l'aire et le périmètre de ce rectangle.
4. Écrivez une classe qui contient un champ privé pour un nom d'utilisateur et un autre champ privé pour le mot de passe associé. Écrivez des méthodes publiques pour définir et obtenir ces champs.
5. Écrivez une classe qui contient un champ privé pour stocker un nombre entier. Écrivez une méthode publique pour augmenter ce nombre de 1 et une autre méthode publique pour diminuer ce nombre de 1.

Exercice 4 (champs de class et champs d'objet) :

1. Écrivez une classe Person avec un champ de classe static counter qui compte le nombre d'instances créées de cette classe.
 - a. Comment pouvez-vous accéder au champ counter depuis une méthode non-static de la classe Person ?
 - b. Comment pouvez-vous accéder au champ counter depuis une autre classe qui n'est pas dans le même paquetage que la classe Person ?
2. Écrivez une classe Calculator avec un champ de classe static pi qui stocke une valeur approximative de Pi.
 - a. Comment pouvez-vous accéder au champ pi depuis une méthode non-static de la classe Calculator ?
 - b. Comment pouvez-vous modifier la valeur du champ pi depuis une méthode non static de la classe Calculator ?
3. Écrivez une classe Employee avec un champ de classe static averageSalary qui stocke la moyenne des salaires des employés de cette entreprise.
 - a. Comment pouvez-vous accéder au champ averageSalary depuis une méthode non static de la classe Employee ?
 - b. Comment pouvez-vous initialiser la valeur du champ averageSalary lors de la création de la première instance de la classe Employee ?
4. Écrivez une classe Game avec un champ de classe static highScore qui stocke le score le plus élevé atteint dans le jeu.
 - a. Comment pouvez-vous accéder au champ highScore depuis une méthode non-static de la classe Game ?
 - b. Comment pouvez-vous mettre à jour le champ highScore si un nouveau score plus élevé est atteint ?
5. Écrivez une classe Shape avec un champ de classe static totalArea qui stocke la somme des aires de toutes les instances de la classe Shape créées jusqu'à présent.
 - a. Comment pouvez-vous accéder au champ totalArea depuis une méthode non-static de la classe Shape ?
 - b. Comment pouvez-vous mettre à jour le champ totalArea lorsque vous créez une nouvelle instance de la classe Shape ?